

Project Report

1. Core Requirements

Key Functional Requirements:

Signal Routing: Support for 2 signal inputs (IN1, IN2) and 16 signal outputs.

Exclusive Output Mapping: A single output channel can only be assigned to one unique input at any given time.

User Interface: Physical control via a single rotary encoder with a built-in push button for channel navigation and selection.

Visual Feedback: A dedicated display module showing the current routing matrix. The currently edited channel must be highlighted. Optional LED indicating device operation.

24/7 Operation & Screensaver: The system must run continuously. To prevent display degradation, a software-based screensaver must be implemented.

OPC Server: Support for OPC clients.

Log System: Generating logs during device usage, whether physically or via a server and recording operation time.

2. Display Choice

Display Technology Comparison		
OLED	LCD	E-Paper

Pros	Good contrast, viewing angles, fast response time, good-looking	Immune to burn-in, very reliable, cheap.	Paper-like readability, no burn-in, esthetic
Cons	Highly susceptible to burn-in, requires a software screensaver.	Narrower viewing angles, requires a backlight	Slow refresh rate
Conclusion	Currently in use; the screen saver isn't a major issue, a proven choice.	Also a good choice, suitable for frequent use, decent appearance.	It is completely unsuitable for a dynamic UI controlled by a rotary encoder.

Based on the required aspect ratio and 24/7 operational constraints, the following specific models were evaluated:

3.12" OLED 256x64

- used in previous equipment, tested,
- screen saver isn't a major issue
- sufficient size
- acceptable price

[Wyświetlacz OLED 3,12" 256x64px niebieski - Sklep msalamon](#)

2.23" OLED 128x32

- smaller one

<https://botland.com.pl/raspberry-pi-pico-hat-klawiatURY-i-wyswietlacze/19539-wyswietlacz-oled-223-128x32px-spil2c->

[czarno-bialy-do-raspberry-pi-pico-waveshare-19750-5904422371937.html](https://botland.com.pl/wyswietlacze-ips/22734-wyswietlacz-lcd-ips-19-170x320px-spi-262k-waveshare-23822.html)

IPS LCD 1,47" 172x320px/LCD IPS 1,9" 170x320px

- Color-Coded User Interface
- true 24/7 operation, allowing the routing matrix to remain continuously visible without the need for a software screensaver.
- an interesting alternative

<https://botland.com.pl/wyswietlacze-ips/22734-wyswietlacz-lcd-ips-19-170x320px-spi-262k-waveshare-23822.html>

<https://botland.com.pl/wyswietlacze-ips/22459-modul-z-wyswietlaczem-ips-lcd-147-172x320px-dfrobot-dfr0995-6959420923014.html>

2. Operating System Selection

Selecting the correct Operating System for the Raspberry Pi Zero W v1 is critical due to its hardware limitations.

My proposal: Raspberry Pi OS Lite (32-bit)

Justification: The ARMv6 processor on the original Pi Zero W does not support 64-bit instructions. A 32-bit OS is strictly required.

The "Lite" version does not include a Desktop Environment (GUI/X11/Wayland). Since the device relies entirely on a dedicated SPI/I2C OLED screen and a rotary encoder, a desktop environment is unnecessary and would consume critical CPU cycles and RAM.

I/O Expander: MCP23017

Encoder suggestions:

<https://www.tme.eu/pl/details/pec11r-4215f-s0024/enkodery-inkrementalne/bourns/>

<https://botland.com.pl/enkodery/9533-czujnik-obrotu-impulsator-enkoder-obrotowy-dfrobot-ec11-6959420912414.html>

<https://botland.com.pl/enkodery/6903-enkoder-z-przyciskiem-20-impulsow-12mm-ec11-pionowy-5szt-5903351245425.html>

3. System Architecture and Implementation

The application is structured as a small set of cooperating modules, each responsible for one hardware resource or one cross-cutting concern. This separation made it possible to diagnose and fix the reliability issues described in Section 4 one subsystem at a time.

3.1 Main Control Loop (main.py)

The entry point runs a single, non-blocking main loop rather than using threads for hardware polling. Each iteration performs, in order: a systemd watchdog ping, encoder polling, lock-timeout checking, screensaver timing, relay routing updates, and conditional re-rendering of the display.

Design decisions were driven directly by 24/7 operation requirements:

- **Systemd watchdog integration (sdnotify).** The loop calls `notify_watchdog()` at the very start of every iteration. If any

downstream call blocks indefinitely (e.g. a stalled I2C/ SPI transaction), the watchdog ping simply stops arriving and systemd kills and restarts the process after a configured timeout (`RuntimeWatchdogSec=15`). This is a safety net for hangs that do not raise a Python exception.

The loop also implements adaptive polling: during the screensaver state, encoder polling and lock-timeout checks are throttled to lower frequencies (10 Hz and 5 Hz respectively) since no user interaction is expected, reducing unnecessary I2C traffic and CPU wake-ups.

3.2 Display Management (`screen_manager.py`)

ScreenManager wraps the JD9853 driver and adds two reliability features on top of the vendor driver:

- **Partial (tile-based) updates.** Each rendered frame is diffed against the previous frame in 40x40 pixel tiles; only tiles that changed are sent over SPI, and adjacent changed tiles are merged into larger rectangular regions to minimise the number of SPI transactions. If more than 55% of the screen changed, the manager falls back to a full-frame update, which is cheaper than sending many small regions.
- **Display reconnect logic.** If the SPI/LCD initialisation fails at startup (or the connection is lost during operation), `disp` is set to `None` and `show()` becomes a no-op rather than raising. The main loop periodically (every 5 seconds) attempts `display.connect()` again, so the system can recover automatically from a transient display fault without manual intervention or a full process restart.

3.3 Lock Manager - Local/Remote Arbitration (`lock_manager.py`)

Because both the physical encoder (main thread) and OPC UA clients (background thread, see 3.4) can request control of the routing matrix, LockManager mediates exclusive access with an owner field, a configurable inactivity timeout, and a priority flag that determines whether local or remote control wins when a conflict occurs.

A `threading.Lock` protects every state-changing operation (`try_acquire`, `release`, `force_steal`, `check_timeout`). This was added after review identified that the OPC UA server thread and the main loop could race on the owner field - for example, a timeout clearing the lock in one thread while another thread was mid-way through granting it - which could allow a brief window of inconsistent access control under real client traffic.

3.4 OPC UA Server (`opc_server.py`)

The OPC UA server runs `asyncua` in a dedicated daemon thread with its own event loop, exposing read-only status variables (current routing, lock owner, recent log lines) and two methods, `RouteSignal` and `ReleaseLock`, that remote clients use to request routing changes under the same LockManager arbitration used by the physical encoder.

The server thread's run loop is wrapped in a restart supervisor: if the `asyncua` server raises an exception (e.g. a transient network issue, or an exception raised inside a client-invoked method), the exception is logged with a full traceback and the server is re-instantiated and restarted after a short delay, rather than allowing the thread to terminate silently. Individual OPC method handlers

(RouteSignal, ReleaseLock) also catch and log their own exceptions so a single malformed client request cannot take down the whole server thread.

3.5 GUI State Machine (gui_matrix.py)

The interface is implemented as an explicit state machine (MENU, ROUTING, ROUTING_GRID, SETTINGS, COLOR_SELECT, STEAL_PROMPT, SCREENSAVER) rendered with Pillow. Screen transitions are driven by encoder rotation (cursor movement) and click events (selection), with the state machine also handling access-control feedback - for example, showing a "lock held" warning or a steal-lock confirmation prompt when a user attempts to modify routing while another party holds the lock.

3.6 Output Matrix / Relay Tree (Expander/relay_board.py)

The physical switching stage does not use one dedicated relay per (input, output) pair - with 2 inputs and 16 outputs that would require 32 independently addressable relays. Instead, RelayBoard implements the routing as a 4-level binary decision tree with crossover stages, so that the two inputs share the same physical relay stages, and only 22 control lines (driven through an MCP23017 I2C GPIO expander) are needed to address all 32 possible input-to-output paths.

Selecting an output channel (1-16) is treated as a 4-bit number (0-15). Each bit of that number selects the direction (one of two branches) at one level of the tree:

```
val = output_ch - 1
b3 = (val >> 3) & 1 # level 1 (top split)
b2 = (val >> 2) & 1 # level 2
```

```
b1 = (val >> 1) & 1 # level 3
b0 = val & 1 # level 4 (crossover output stage)
```

`_calculate_routing_path()` then maps each of the four levels to a specific logical control line (`CTRL_1 .. CTRL_22`) and a target state (on/off). Input 1 and input 2 use disjoint sets of control lines for levels 1-3 (`pin_lvl1/2/3` are offset by a fixed base - 1, 2+b3, 4+... for input 1; 8, 9+b3, 11+... for input 2), reflecting the fact that each input enters the tree from a different physical side. Level 4 - the final crossover stage - is shared: both inputs can reach the same physical output relay pair (`pin_lvl4` is computed identically for both), but the desired state `s4` is inverted between the two inputs (`bool(b0 ^ 1)` for input 1 vs `bool(b0)` for input 2), which is what lets either input reach either half of the last stage without a dedicated relay per input.

Atomic hardware write. All pin changes for a routing update are first applied to an in memory bitmask (`_set_logical_pin`) and only written to the MCP23017 once, via a single `flush_state()` call that writes both GPIO banks. This avoids sending the relay matrix through intermediate, partially-updated states over I2C.

Configuration values referenced throughout this section - I2C addresses for the MCP23017 and the encoder, display dimensions and orientation, and filesystem paths - are centralised in `Config/settings.py`, so that hardware-specific constants are not duplicated across modules.

3.7 Runtime Tracking and Unified Event Logging

Two further additions were made to support long-term maintenance and auditing, addressing a requirement to record

cumulative operation time and to make the log file readable by someone unfamiliar with the codebase.

RuntimeTracker (Utils/runtime_tracker.py) maintains a persistent, crash-safe counter of total device operation time, independent of the current process's uptime. Rather than writing to disk on every loop iteration - which would be unnecessary SD card wear - elapsed time is accumulated in memory and flushed to a small JSON file at most once per minute, using the same atomic write pattern (write to a temporary file, then `os.replace()`) used elsewhere in the project for crash-safety. In the worst case, an unclean shutdown loses at most one flush interval of runtime; the counter is otherwise unaffected by process restarts or kernel-level failures. Total runtime is exposed in three places: an hourly [RUNTIME] log entry, a dedicated OPC UA variable (Total_Runtime_Hours) for remote monitoring, and a read-only line on the on-device Settings screen.

Event logging was also standardised around a small set of fixed tags, so that the log file can be filtered or grepped without needing to read the source: [ROUTE] for every routing change (recording its source - LOCAL or a client identifier - and the input/output pair), [LOCK] for every access-control event (acquired, released, timeout, stolen), and [RUNTIME] for periodic uptime snapshots. Previously, routing changes were logged as a verbose, per pin string intended for low-level hardware debugging; this has been replaced with a single, consistent line per event aimed at operational review rather than hardware debugging.

3.8 Network Status and Offline-Safe Clock

The device is required to operate fully without network access, which complicates showing a meaningful time in logs: the Raspberry Pi Zero W has no battery-backed real-time clock, so an offline boot has no reliable notion of the current time.

Utils/network_info.py provides a single, cached IP lookup (hostname -I, refreshed at most every 15 seconds) shared by three consumers: the status bar (shown centred on every screen, used to locate the device for an OPC UA connection), and Utils/device_clock.py, which uses the presence of any IP as a heuristic for "the OS clock is NTP-synced" - a deliberate simplification, since correctness here only needs to be as good as Raspberry Pi OS's own automatic time sync, not internet connectivity.

device_clock exposes a best-effort wall clock for log timestamps only: when online, it defers directly to the OS clock; when offline, it reports an explicit "unknown" state (rendered as --:-- in logs) until the operator sets the time manually through a new Settings screen (Set Time), edited one field at a time with the encoder. The manually set time is kept purely in memory as an offset from a monotonic clock reading and intentionally does not persist across restarts - an accepted trade-off given the lack of an RTC, rather than persisting a value that would silently drift further from reality with every uncorrected restart. If the device regains network access later in the same session, timestamps automatically revert to the OS clock with no restart required, since the online/offline check is re-evaluated on every read.

Utils/logger.py was updated with a custom DeviceClockFormatter that sources the asctime field of every log line from device_clock instead of the raw OS clock, so an offline, unsynced device reports its own uncertainty honestly rather than printing a plausible-looking but fabricated timestamp.

4. 24/7 Reliability Engineering

4.1 Problem Statement

During initial continuous-operation testing, the system stopped responding after approximately 6 hours of uninterrupted use. The application logs showed no exception and no error prior to the failure - the process simply disappeared and systemd restarted it. This pointed to a failure below the Python application layer, most likely in the Linux kernel or a device driver.

4.2 Diagnostic Infrastructure

Before the root cause could be identified, the following diagnostic capabilities had to be put in place:

- **Hardware watchdog** (systemd RuntimeWatchdogSec + dtparam=watchdog=on) - ensures the board recovers automatically from a hard hang rather than staying unresponsive indefinitely.
- **Persistent kernel journal** (journald Storage=persistent, with a bounded SystemMaxUse to protect the SD card, and a short SyncIntervalSec so that log entries are flushed to disk before a hard reset can lose them) - required because Raspberry Pi OS ships with a default drop-in configuration that forces volatile (RAM-only) journal storage, which was silently overriding the persistent setting until identified and overridden with a higher priority drop-in file.

With these in place, the next failure produced a complete kernel oops trace rather than an empty log.

4.3 Root Cause Analysis

The captured kernel oops showed:

Unable to handle kernel execution of memory at virtual
address ... when execute

Internal error: Oops: 8000000f [#1] ARM

Comm: python3 ... Tainted: G D C 6.18.34+rpt-rpi-v6

LR is at arch_sync_dma_for_device+0x50/0x64

The programme counter had jumped into a kernel stack region rather than valid executable code, with the link register pointing into `arch_sync_dma_for_device` - the DMA cache synchronisation routine invoked by the `spi_bcm2835` driver when preparing a buffer for hardware DMA transfer. The faulting process was consistently `python3`, coinciding with SPI writes to the LCD (the application's only large, frequent DMA-eligible transfers).

This pattern - identical fault location, identical driver, always during an SPI write from the display code - was reproduced across multiple independent long-running tests, which ruled out a one-off hardware glitch and pointed to a stability issue in the `spi_bcm2835` driver's DMA path on this kernel version.

4.4 Resolution

The Raspberry Pi's BCM283x SoC exposes two independent SPI controllers: the primary controller (`spi0`, driven by `spi_bcm2835`) and the auxiliary controller (`spi1`, driven by a separate driver, `spi_bcm2835aux`). Since the fault was isolated to the primary controller's DMA path, the display was moved to the auxiliary bus:

-

Hardware: SCLK, MOSI and CS re-wired from the SPI0 pins to the SPI1 pins (GPIO 21/20/18); the backlight PWM output was moved from GPIO18 to GPIO12 to resolve the resulting pin conflict.

-

Firmware: `dtoverlay=spi1-1cs` added to `config.txt` to enable the auxiliary controller. •

Application: `spidev.SpiDev(0, 0)` changed to `spidev.SpiDev(1, 0)` in the display driver; no other code changes were required, as the `spidev` API is identical across both buses.

This change resolved the fault: the system subsequently ran for over 48 continuous hours without a single kernel oops, compared to a previous maximum of roughly 6 hours before failure. Testing is ongoing to confirm stability over multi-day periods.

4.5 Known Limitations

-- One limitation was identified and deliberately left unaddressed rather than risking a late change: log file rotation reads the raw OS clock directly to decide when to roll over and how to name the resulting file, independently of the `DeviceClockFormatter`. If the device boots offline with an unset or incorrect system date, the timestamps printed inside each log line will honestly read `--:-` - (via `device_clock`), but the rotated file's name (e.g. `signal_splitter.log.YYYY-MM-DD`) may still reflect the OS's own, potentially incorrect, notion of the date. Correcting this would require reimplementing log rotation against the custom clock rather than relying on the standard library's handler, which was judged out of scope for the current release.

-- The SPI1-based fix was validated on a prototype using direct wiring to the Raspberry Pi's GPIO header. The currently designed PCB layout routes only the primary SPI bus (SPI0) to the display connector; the auxiliary bus (SPI1) is not currently broken out to that connector. Deploying this fix on the fabricated PCB as currently designed would require either a board revision

routing SPI1 to the display connector, or a manual patch wire bridging the relevant GPIO18/20/21 pins directly from the Pi's header to the display connector.

5. Access Control (Lock) State Machine

Both the physical rotary encoder and OPC UA clients must hold a lock before changing routing. The lock has a single owner at a time, a 30-second inactivity timeout, and a priority flag controlling whether local physical control is allowed to override a remote client.

A key asymmetry in the design: only the local encoder can ever steal the lock, and only when priority is set to LOCAL - this triggers a confirmation screen (STEAL_PROMPT) before forcefully taking over. A remote OPC UA client can never steal the lock under any priority setting; RouteSignal() simply returns an "Access Denied" response if the lock is held by someone else. The priority flag is consulted exclusively in the GUI layer and is never read inside `opc_server.py`.



